Fathom

Every verdict here is computed in code.

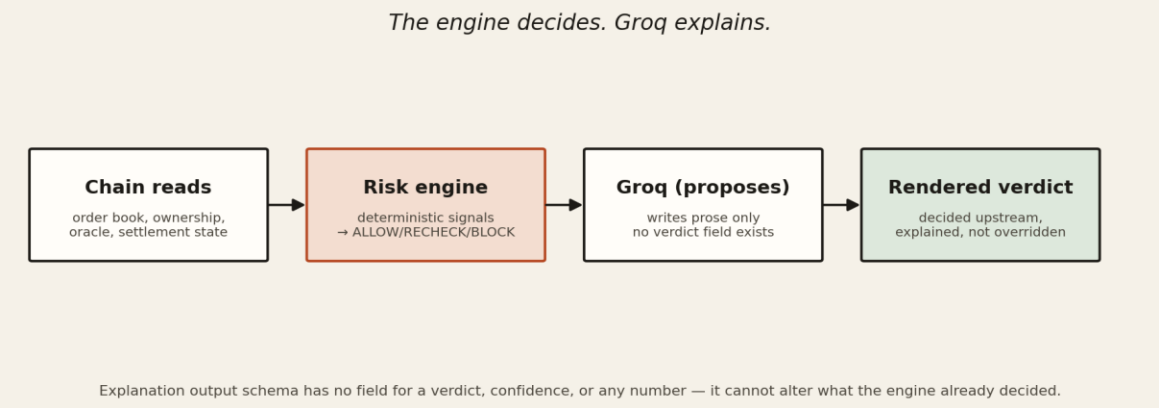
Technical Proof of Work for Judges · fathom.samuelyahaya.com · Somnia × DreamDEX Event Contracts Hackathon
Submitted September 11, 2026

What Fathom does

Fathom is a risk-verdict overlay for DreamDEX Event Contracts on Somnia's Shannon testnet. For each market, it extracts live order book, trades and oracle state from the blockchain and computes eight deterministic risk signals from it. Each market gets one verdict: ALLOW, RECHECK or BLOCK. A language model then explains that verdict in plain English. The output schema of the model has no place for a verdict, for a confidence level and for any numerical value which would override the engine's decision. Most apps provide you with a price and ask you to trust it. Fathom tells you whether that price can be trusted at all.

Architecture: the engine decides, Groq explains

The risk engine is deterministic and makes no model call. Every signal (liquidity, depth durability, volatility, staleness, window, order flow, resolution and venue) is computed from data read on-chain and from the venue before the language model is called at all. The model then receives the finished verdict and the measured signals, and returns prose: it explains what the engine found, and has no way to change it.



Two invariants worth a second look

1. The book is read twice, and it is allowed to disagree with itself

Every venue interface shows "the book, as displayed": bids, asks, and a mid price assembled from the indexer's materialized rows. Fathom also assembles "the same book, as owned", from a separate on-chain call that returns the owner and expiry timestamp of every resting order (getAllOpenOrdersOffChain). These fields exist on-chain but are summed away by every aggregated view, including the venue's own indexer. On this venue the two readings can describe very different books: a displayed price backed by depth that is, on inspection, entirely owned by one address and withdrawable at will. Fathom states both readings side by side rather than collapsing them into one number, because collapsing them is exactly what hides the risk.

Real fields, read directly: owner, expireTimestampNs, present on-chain, absent from the materialized book and from the indexer's rows.

2. A verdict cannot be represented as model output

There is no field in the schema for the explanation step to which a model could write a verdict, a severity, or a number that had not been calculated by the engine already. It is an invariant at the level of types, in the sense of “invalid states are unrepresentable”; the model is not just told not to overwrite the verdict but is not provided with a place where it could do so.

This invariant was tested rather than simply assumed. A schema drift bug (see below) made the model's path fail entirely; that the engine went on computing correct verdicts throughout is what shows the invariant holds.

Verification: bugs caught by actually using it

The following five observations were made based on the output of the program, rather than from the analysis of the source code itself. Four are bugs in the software; one is a corrected assumption concerning the venue itself, perhaps the most significant of the five, since it was an incorrect assumption to begin with.

Finding 1: a gate that could pass while broken

A check asserting every market was explained by the model had no assertion on explanation.source. A hand-copied list of signal ids in the model's schema went stale the moment a new signal was added, strict decoding rejected every response, and three markets fell back to the deterministic narrator. The gate still reported full success:

```
explained by deterministic fallback   (x3)
gate
  PASS — model explains, engine decides, guard enforces      exit 0
```

A second gate, grading the live board, passed with zero markets graded when pointed at an invalid venue id: every assertion loop over an empty result set was vacuously true:

```
summary        ALLOW 0  RECHECK 0  BLOCK 0  of 0
determinism    PASS same snapshot, same verdict
discrimination PASS 0 signal(s) vary across markets (none)
               PASS severity → verdict mapping holds on all 0 markets
confidence     min Infinity max -Infinity across 0 markets
gate           PASS — deterministic, discriminating, confidence tracks observability
```

Fix: both gates now fail on an empty board, an ingest failure, or a mismatch between explanation.source and the run mode, proven by deliberately reintroducing each failure condition and confirming the gate now catches it rather than merely reading correct on a happy path.

Finding 2: a failed read presenting as a measurement

The freshness property was tied up in an unconditional “ok” provenance label. Whenever the fill properties showed a degraded status and the index row did not have a lastTradeAt field, the trace still showed “this market has never traded” with high severity, being labelled as a true reading rather than as a failed read, which is the single failure case that the product’s entire provenance scheme seeks to avoid. It was a failure condition that remained obscure, as most of the board really hadn’t ever traded.

Solution: the creation of a recencyUnknown status, mutually exclusive with neverTraded, so a degraded read is now reported as unmeasured rather than as a false observation.

Finding 3: a corrected assumption, not a corrected typo

The depth durability test was supposed to indicate whether the resting order's owner had the ability to cancel the order, based on finding the cancel selector in the bytecode of the owner. In this case, each order owner was a 291-byte proxy which called

an upgradeable beacon's implementation. That is, the bytecode of the owner did not contain any selectors at all and “no cancel selector found” would be incorrectly understood as “cannot cancel.” There was a cancelOrder function in the beacon's implementation, and the beacon was also upgradeable by some address.

Correction: a resting order owned by a proxy could not be certified as non-pullable, because the bytecode controlling the ability to cancel belonged to the address which controlled the beacon, not the order owner. Even then, firmness was limited only to “firm-until-expiry,” and even that required checking of an additional capability.

This is the finding we are most confident is correct, precisely because it survived being wrong first.

Finding 4: a countdown that had stopped

The board is served from a committed capture, and every row printed the time left before its market expired. That figure was a delta measured at capture and then reprinted forever, so the deployed page said expires 54m for markets that had settled hours earlier: seven rows out of seven, each wrong independently. A banner reading captured 12h ago does not repair it, because a figure in a table reads as current whatever the header says.

Fix: the row no longer carries a countdown, it carries the instant the market expires, and counts against a clock that ticks. The gate gained the case that actually shipped broken, a row live at capture that has expired since; reverting the logic makes it fail all three new assertions.

Finding 5: a pin that became a time bomb

The stuck market below could not be reached through the normal market list, so it was pinned into the capture by id, and the capture refused to write a board that was missing it. That was correct while the market existed. When it was voided the id left the venue registry, and every capture then refused rather than write a board without it: two scheduled runs and a manual dispatch all declined, and the board sat nine hours stale before anyone looked.

Fix: the pin is empty and the reason is recorded where it happened. A required-market pin on something that can disappear is not a row that might go missing, it is a stop on the refresh path; if one is ever added back it needs an expiry, or its absence has to be a warning rather than a hard exit.

A real case, start to finish: the market that outlived its indexer

A market on this venue had its settlement window close and then sat unresolved for ten days, its collateral locked, while the indexer went on reporting it as trading. It was captured while that was true. It is no longer true, and both halves are on chain:

```
market contract  0x27f6DE3dbdb1fc0A4E35B6b187A28c25B4B49867
market id        0x000000000000000000000000000000000000000000000000000000000000c067

as captured 2026-09-02 00:59 UTC
  status()   = 2 (Locked)     isResolved = false    isVoided = false
  backing()  = 1,503,000,000 raw = 1,503 tUSDC locked
  expiry 2026-08-28 15:00 UTC + 300s window; voidExpired() callable, uncalled
  indexer clobStatus = "Trading"   — throughout

re-read 2026-09-08 16:20 UTC, block 483,115,391
  status()   = 5 (Voided)     isVoided = true       backing() = 0
  voidExpired() called 2026-09-08 12:30:59 UTC, block 482,974,680
                10d 21h after it became callable
```

An interface trusting the indexer's status field would have shown this market as actively trading for all ten days. Fathom read chain state directly and graded it BLOCK on a not-trading / cannot-settle basis. Because the engine's default market list filters to unexpired markets, the case could not surface on its own; it was pinned into the capture by id and shown flagged as past its expiry rather than folded silently into a normal row.

Then someone called voidExpired(), and the evidence ended. That was always the risk: the call is permissionless, which is why the read was frozen at fixtures/stuck-market-c067.json the day it was found rather than cited from a live query. The fixture is still graded by the test suite, so the case survives as a regression test after the market itself is gone, and both halves are checkable on chain at the addresses above. It is the clearest evidence in the product that reading the chain rather than the indexer is not a stylistic preference: it is the only reason this was catchable at all.

The board refreshes itself, and refuses to make itself worse

The dashboard serves a committed capture rather than reading the venue per request: one live pass is about 150 round trips, measured at 46 seconds, which does not fit a serverless request budget. Because the capture is a committed file that the app bundles by static import, refreshing the board and deploying it are the same action, so an hourly GitHub Actions job runs the capture and pushes the result. No datastore, no scheduler service, no new runtime failure mode.

It fails closed, three ways. A pass that cannot reach a market it was told to include writes nothing. A pass producing fewer distinct verdicts than the floor writes nothing, so an unattended job cannot quietly replace a board showing ALLOW, RECHECK and BLOCK with one showing only RECHECK. A pass that reproduces the current board byte for byte costs no deploy. The floor is not a constant either: past four hours the board's staleness outranks its spread and the floor drops to two, because a board whose rows have all expired demonstrates nothing whatever its tally says.

Measured, it is a backstop and not a guarantee. Across the first seven hourly slots GitHub fired one, 39 minutes late, and silently dropped the rest; scheduled runs are documented as best-effort under load, and that is a coin flip per slot rather than an occasional hiccup. So the board is still captured by hand before anything is recorded, and the page states its own read age instead of implying it is live.

Why this works

The principle underneath Fathom is not novel to crypto: verify against the primary record, not a secondary account of it. Financial audits and investigative reporting both run on the same discipline, a ledger entry outranks a summary of the ledger; a source document outranks a press release describing it. Prediction-market interfaces have, so far, mostly trusted the indexer's summary. Fathom checks the ledger.

Alignment with judging criteria

Technical Implementation (25%)
Real SDK depth: on-chain order-book reads, per-order ownership resolution through proxy delegation, settlement-window-aware lapse calculation, and a gate suite that fails on real regressions rather than passing on a happy path alone, plus an unattended hourly capture that fails closed rather than degrade the board it exists to refresh.

Innovation & Originality (20%)
The obvious way to use a model here is to let it decide a trade. Fathom inverts that: the model is schema-constrained to explanation only, and the signal it rests on (depth durability, resolved through proxy-beacon delegation) has no off-chain equivalent anywhere on the venue.

User Experience & Design (20%)
An audit-document interface, not a trading terminal: severity read as ink density rather than a traffic light, a read-only claim stated plainly instead of a dead wallet button, and a detail page that shows its own reasoning rather than asserting a verdict without evidence.

Business & Ecosystem Impact (20%)

Fathom does not create trading activity; it safeguards it. It is the shield that a faster or more leveraged service would use to verify a market's condition before routing an order into it, due diligence below the trading experience, not above it.

Presentation & Demo (15%)

The demo's central proof point is not a claim, it is a pair of on-chain reads at an address printed in full above: the divergence as it was captured, and the void that ended it six days later. A judge can check both, and neither asks anyone to take this document's word for it.

Project timeline

Stage	Outcome
Venue research	Read the Bot Kit SDK and live venue behaviour before writing any risk logic; confirmed indexer/chain divergence was real and reproducible, not assumed.
Core design	Deterministic engine with a schema-constrained explanation step locked before any UI work began; verdict decided upstream of the model, always.
Signal calibration	Every threshold calibrated against this venue's own measured distributions rather than generic market intuition; three signals initially treated as discriminating turned out to be venue constants and were corrected.
Depth-durability signal	Built, found wrong (proxy-beacon delegation), corrected, and re-verified against measured on-chain data twice.
Gate hardening	Found and fixed gates capable of reporting PASS while the path they claimed to test had silently degraded; each fix proven by reintroducing the failure.
Case capture	Located and captured a live, independently verifiable indexer/chain divergence as a reproducible fixture. The market was voided six days later; the fixture outlived it and is still graded by the test suite.
Design & dashboard	Audit-document interface, ink-depth severity, read-only stated directly rather than implied by an absent wallet flow.
Deployment	Fixture-mode render chosen deliberately for serverless correctness, not convenience; documented and labeled honestly rather than presented as continuously live.
Scheduled capture	Hourly GitHub Actions job refreshes the committed board and deploys by pushing it; it refuses to write on an unreachable market, too thin a verdict spread, or no change at all. Measured, GitHub delivered one of seven scheduled slots, so it is treated as a backstop, not a guarantee.
Submission	Live board, demo video, this document, and an SDK/venue feedback report submitted alongside the repository.

Prepared by Samuel Urah Yahaya

GitHub: Sammy949
X: @I_am_SamY01
Live: fathom.samuelyahaya.com
Repository: Sammy949/fathom

September 11, 2026